

Bloc 2

Concevoir et développer des applications logicielles

Dossier de certification — projet CountryTravel

Candidat	Lucas Lafourcade
École	Ynov Campus
Projet support	CountryTravel — plateforme de découverte de destinations de voyage
Code source	github.com/aref400/countryTravel
Livrables joints	Code source et documentation technique (dossier docs/)
Date de remise	Juillet 2026

Sommaire

Sommaire	2
Table de correspondance avec le référentiel	4
1. Présentation du projet	5
1.1 Le produit	5
1.2 Périmètre livré	5
1.3 Logiciel fonctionnel en production	5
2. Architecture logicielle	6
2.1 Vue d'ensemble	6
2.2 Choix structurants pour la maintenabilité	6
2.3 Frameworks et paradigmes	6
3. Prototypage : le moteur de recommandation	7
3.1 Parcours utilisateur	7
3.2 Fonctionnement du scoring	8
3.3 Ergonomie et sécurité du prototype	8
4. Intégration et déploiement continus	9
4.1 Protocole d'intégration continue	9
4.2 Protocole de déploiement continu	9
5. Critères de qualité et de performance	10
5.1 Outils qualité	10
5.2 Couverture de tests — mesures réelles	10
5.3 Performance — mesures réelles (environnement local, base Docker)	11
6. Jeu de tests unitaires : le ScoringService	11
7. Mesures de sécurité (OWASP Top 10)	12
8. Accessibilité	12
8.1 Actions mises en œuvre	12
9. Cahier de recettes	13
9.1 Méthodologie	13
9.2 Exécution	13
9.3 Extrait représentatif	14
10. Plan de correction des bogues	15
10.1 Processus de traitement	15
10.2 Synthèse des anomalies traitées	15
10.3 Exemple de fiche	16
11. Historique des versions	16
12. Manuels	17
12.1 Manuel de déploiement	17
12.2 Manuel d'utilisation	17
12.3 Manuel de mise à jour	17
13. Axes d'amélioration et perspectives	18
13.1 Axes techniques identifiés	18
13.2 Perspectives produit	18
Annexe — Documents complets remis avec le code source	18

Table de correspondance avec le référentiel

Ce dossier couvre l'intégralité des éléments attendus au titre du Bloc 2. Chaque exigence du référentiel d'évaluation est mise en regard de la compétence visée, de la section du dossier qui la traite et du livrable complet remis avec le code source.

Élément exigé par le référentiel	Compétence	Section	Livrable complet (dépôt remis)
Protocole d'intégration continue	C2.1.2	4.1	<code>.github/workflows/pr.yml</code>
Protocole de déploiement continu	C2.1.1	4.2	<code>.github/workflows/deploy-*.yml</code>
Critères de qualité et de performance	C2.1.1	5	<code>docs/qualite-performance.md</code>
Architecture logicielle structurée et maintenable	C2.2.1	2	Code source (monorepo)
Présentation d'un prototype réalisé	C2.2.1	3	Code source (module scoring)
Frameworks et paradigmes de développement	C2.2.1	2.3	Code source
Jeu de tests unitaires sur une fonctionnalité	C2.2.2	6	<code>apps/api/src/scoring/scoring.service.spec.ts</code>
Mesures de sécurité (OWASP Top 10)	C2.2.3	7	<code>docs/securite-accessibilite.md</code>
Actions d'accessibilité et référentiel justifié	C2.2.3	8	<code>docs/securite-accessibilite.md</code>
Historique des différentes versions	C2.2.4	11	Historique Git et pull requests
Dernière version fonctionnelle, fiable et viable	C2.2.4	1.3	URLs de production
Cahier de recettes	C2.3.1	9	<code>docs/cahier-recettes.md</code>
Plan de correction des bogues	C2.3.2	10	<code>docs/plan-correction-bogues.md</code>
Manuel de déploiement	C2.4.1	12.1	<code>docs/manuel-deploiement.md</code>
Manuel d'utilisation	C2.4.1	12.2	<code>docs/manuel-utilisation.md</code>
Manuel de mise à jour	C2.4.1	12.3	<code>docs/manuel-mise-a-jour.md</code>

Périmètre du dossier

Conformément à la limite de 30 pages, ce dossier présente pour chaque exigence la méthode retenue, les décisions structurantes et les résultats mesurés. Les documents intégraux (cahier de recettes, plan de correction, manuels, référentiels sécurité et accessibilité) sont remis avec le code source dans le dossier docs/.

1. Présentation du projet

1.1 Le produit

CountryTravel aide un voyageur à choisir sa prochaine destination. La plateforme repose sur un moteur de recommandation personnalisé — un formulaire multi-étapes produit un classement des cinq pays les plus compatibles — complété par une destination aléatoire, une carte mondiale interactive colorée selon les notes de la communauté, des fiches pays détaillées et un volet communautaire (pays visités, avis notés, tableau de bord personnel).

L'objectif est de proposer une sélection de pays réellement adaptée aux préférences de l'utilisateur, tout en conservant une dimension communautaire portée par les notes et les avis des voyageurs.

1.2 Périmètre livré

Fonctionnalités en production à la date de remise :

- **Authentification** par e-mail et mot de passe (JWT access et refresh)
- **Catalogue de 30 pays** documentés selon des critères de voyage : budget, sécurité, climat et huit « niveaux » thématiques
- **Moteur de recommandation** : score de 0 à 100, filtres éliminatoires puis pondération
- **Sauvegarde des recommandations**, destination aléatoire et carte mondiale interactive
- **Volet communautaire** : pays visités, avis notés (un avis maximum par pays, réservé aux pays visités) et tableau de bord

1.3 Logiciel fonctionnel en production

Environnement	URL
Front (Netlify)	https://grand-stardust-e3ca80.netlify.app/
API (Railway)	https://disciplined-art-production-4eaa.up.railway.app/api/v1

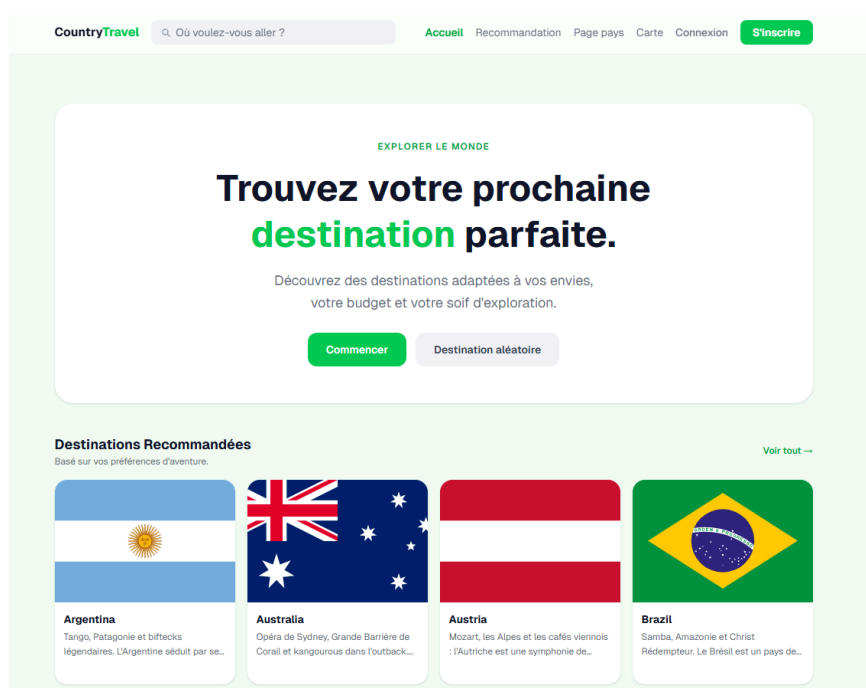


Figure 1 — Page d'accueil de CountryTravel en production

2. Architecture logicielle

2.1 Vue d'ensemble

Le projet est organisé en monorepo npm workspaces réunissant deux applications :

- **apps/api** — NestJS 11, Prisma 7 et PostgreSQL. Le découpage suit les modules métier (auth, countries, recommandations, scoring, visits, reviews), chacun regroupant son contrôleur, son service et ses DTO. La documentation Swagger est générée automatiquement (/api/docs, désactivée en production).
- **apps/front** — React, TypeScript et Vite. L'organisation est faite par fonctionnalité (src/features/* : auth, countries, recommandations, reviews, dashboard) au-dessus d'une couche partagée (src/shared : client HTTP, store Zustand, composants d'interface).

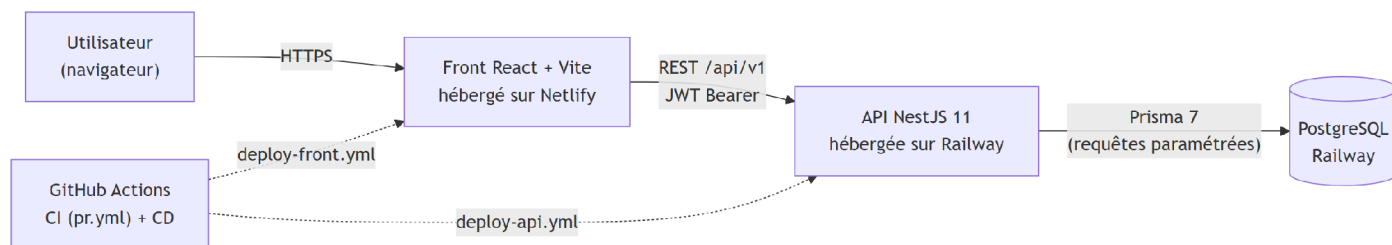


Figure 2 — Architecture technique de CountryTravel

2.2 Choix structurants pour la maintenabilité

- Séparation stricte contrôleur (HTTP), service (métier) et Prisma (accès aux données) côté API
- Validation systématique des entrées aux frontières : class-validator sur les DTO et ValidationPipe global en mode whitelist
- Découpage par feature côté front : chaque fonctionnalité embarque ses composants, hooks, services et schémas, et se comprend comme se modifie sans toucher aux autres
- Type TypeScript strict de bout en bout, y compris sur le client Prisma généré
- Base de données versionnée par des migrations Prisma committées

2.3 Frameworks et paradigmes

Couche	Framework / outil	Paradigmes mis en œuvre
API	NestJS 11	Injection de dépendances, modules, décorateurs, guards et pipes (programmation par aspects)
ORM	Prisma 7	Schéma déclaratif, requêtes paramétrées, migrations versionnées
Front	React 18 + Vite	Composants fonctionnels, hooks, état immuable
État global	Zustand	Store minimal, sélecteurs
Formulaires	React Hook Form + Zod	Validation déclarative par schéma, typage inféré
Styles	TailwindCSS + shadcn/ui	Utility-first, composants accessibles

Côté API, NestJS impose un découpage en **modules métier** (auth, countries, recommandations...), chacun encapsulant son contrôleur HTTP, son service métier et ses DTO validés, l'accès aux données restant isolé derrière Prisma. Un module se teste et évolue seul : c'est le pendant back du découpage par feature du front, la même logique de maintenabilité appliquée aux deux applications.

Côté front, le découpage est **par fonctionnalité** (src/features/*) plutôt que par type technique : chaque feature embarque ses composants, hooks, services et schémas, et se modifie sans risquer d'en casser une autre. Un compromis est assumé : le module auth, le plus sensible, colocalise ses pages et ses routes là où les autres pages vivent dans src/pages. Ce choix est délibéré et son harmonisation est identifiée comme axe d'amélioration.

3. Prototype : le moteur de recommandation

La fonctionnalité retenue au titre de la compétence **C2.2.1** est le parcours « formulaire multi-étapes → top 5 recommandé », cœur métier du produit.

User stories couvertes par le prototype :

En tant que...	je veux...	afin de...
voyageur	répondre à un questionnaire de préférences	obtenir des destinations adaptées
voyageur	voir un top 5 scoré (0–100)	comparer objectivement les destinations
voyageur	ouvrir la fiche d'une destination proposée	approfondir avant de choisir
utilisateur connecté	sauvegarder une recommandation	la retrouver dans mon tableau de bord
voyageur pressé	obtenir une destination au hasard	me laisser surprendre

3.1 Parcours utilisateur

Étape 1

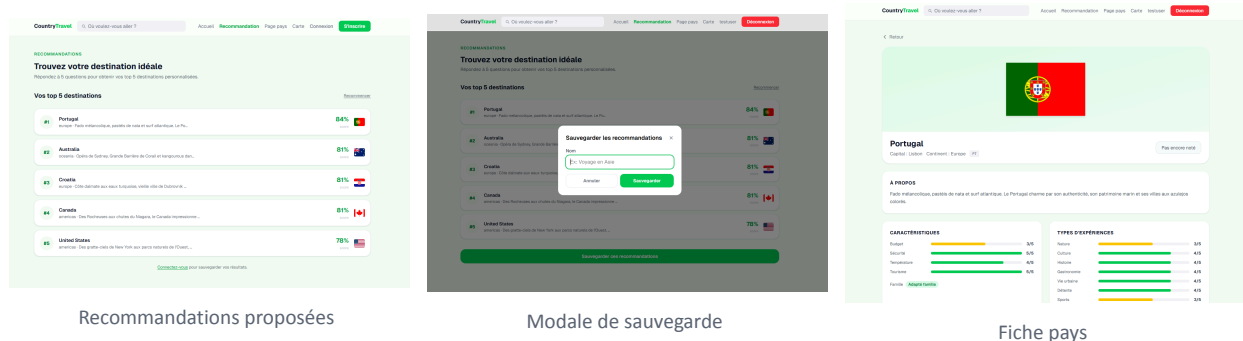
Étape 2

Étape 3

Étape 4

Étape 5

Figure 3 — Les cinq étapes du formulaire de recommandation



Recommandations proposées

Modale de sauvegarde

Fiche pays

Figure 4 — Restitution du top 5, sauvegarde et accès à la fiche pays

3.2 Fonctionnement du scoring

- Filtres éliminatoires** — les pays incompatibles avec les critères non négociables (sécurité minimale, par exemple) sont exclus avant tout calcul.
- Score pondéré de 0 à 100** — chaque critère du formulaire (budget, climat, affinités de voyage) contribue au score selon sa pondération.
- Top 5** — les pays sont classés et les cinq premiers sont retournés avec leur score et leur rang.

Le service **ScoringService** est volontairement **pur et injectable** — il n'accède pas à la base de données. C'est cette propriété qui le rend exhaustivement testable (voir section 6).

3.3 Ergonomie et sécurité du prototype

- Formulaire en cinq étapes avec état conservé : le retour arrière ne perd aucune saisie

- Parcours utilisable sans compte grâce à un guard JWT optionnel ; seule la sauvegarde exige l'authentification
- Temps de réponse mesuré à 57 ms en local pour un calcul complet (voir section 5.3)
- Interface responsive : l'application s'adapte du grand écran au mobile, le prototype reste pleinement utilisable sur les équipements web et mobiles

4. Intégration et déploiement continu

4.1 Protocole d'intégration continue

Chaque pull request vers main déclenche le workflow pr.yml (GitHub Actions) :

4. Installation reproductible (npm ci, Node 20, cache npm) et génération du client Prisma
5. **Lint API** (ESLint) — échec bloquant
6. **Tests API avec couverture** (test:cov) — les seuils de couverture (80 % des instructions, 65 % des branches, 75 % des fonctions, 80 % des lignes) sont vérifiés par Jest : passer en dessous fait échouer le job
7. **Build API** — compilation TypeScript stricte
8. **Lint, tests (Vitest) et build (Vite) du front**

Une pull request ne peut être fusionnée que si l'ensemble de la chaîne passe. Le seuil de couverture est **bloquant** : toute baisse en dessous des seuils fait échouer la CI (voir section 5.2).

4.2 Protocole de déploiement continu

Tout merge sur main déclenche :

- **deploy-api.yml** — build et déploiement de l'API sur Railway (PostgreSQL managé)
- **deploy-front.yml** — build Vite et déploiement du front sur Netlify

Point de vigilance documenté

Les migrations de base de données ne sont pas appliquées par le pipeline : la procédure reste manuelle et est décrite dans le manuel de mise à jour (section 12.3). L'axe d'amélioration correspondant — l'ajout d'une étape prisma migrate deploy — est identifié et documenté.

All workflows				Q Filter workflow runs
Showing runs from all workflows				
83 workflow runs		Workflow ▾	Event ▾	Status ▾ Branch ▾ Actor ▾
✓	Correction via npm audit du problème critique PR Checks #38: Pull request #32 opened by aref400	fix/Vulnerabilite-package	32 minutes ago 1m 34s	...
✓	Merge pull request #31 from aref400/fix/BUG-011 Deploy API to Railway #23: Commit a6d188a pushed by aref400	main	Today at 19:37 2m 57s	...
✓	[BUG-011] Ajout de la verification du JWT-REFRESH-SECRET n'est pas null PR Checks #37: Pull request #31 opened by aref400	fix/BUG-011	Today at 19:35 1m 27s	...
✓	Merge pull request #30 from aref400/amelio-code Deploy API to Railway #22: Commit e9af18b pushed by aref400	main	Jul 17, 15:56 GMT+2 2m 59s	...
✓	Merge pull request #30 from aref400/amelio-code Deploy Front to Netlify #22: Commit e9af18b pushed by aref400	main	Jul 17, 15:56 GMT+2 2m 31s	...
✓	Grosse amélioration de code + création du document bloc 2 PR Checks #36: Pull request #30 opened by aref400	amelio-code	Jul 17, 15:51 GMT+2 1m 38s	...

Figure 5 — Exécution de la chaîne d'intégration continue sur GitHub Actions

5. Critères de qualité et de performance

Cette section présente une synthèse. Le détail complet, les seuils et les séquences de vérification figurent dans docs/qualite-performance.md.

5.1 Outils qualité

TypeScript en mode strict, ESLint, Prettier, tests Jest côté API et Vitest associé à React Testing Library côté front, CI bloquante sur le lint et les tests, revues systématiques par pull request.

5.2 Couverture de tests — mesures réelles

- **API : 85 % des instructions, 85 % des lignes, 79 % des fonctions, 71 % des branches**, après exclusion du code généré (client Prisma) et du câblage des modules. Les seuils bloquants en CI sont fixés à 80 / 65 / 75 / 80.
- **Front : stratégie assumée** — les tests unitaires ciblent la couche logique (shared/services à 94 %, shared/lib à 81 %, store et hooks) ; les composants de présentation sont couverts par la recette manuelle, soit 125 scénarios. La couverture globale du front (14 %) n'est volontairement pas seuillée : ce chiffre et la décision qui le sous-tend sont documentés et justifiés dans docs/qualite-performance.md.

Matrice de couverture par couche — les tests unitaires ciblent la logique métier, couverte de 80 à 94 % des deux côtés ; les composants de présentation relèvent de la recette manuelle :

Couche	Outil	Couverture	Nature
API — logique métier (services)	Jest	85 % instr. / 79 % fonctions	unitaire, seuil CI ≥ 80 %
Front — logique (shared/services)	Vitest	94 %	unitaire
Front — utilitaires (shared/lib)	Vitest	81 %	unitaire
Front — store & hooks	Vitest	tests ciblés	unitaire

Couche	Outil	Couverture	Nature
Front — composants de présentation	—	recette (125 scénarios)	fonctionnel

5.3 Performance — mesures réelles (environnement local, base Docker)

Endpoint	Temps de réponse
GET /countries — liste paginée	10 ms
GET /countries/:iso — détail	8 ms
GET /countries/map/all — carte	6 ms
POST /recommendations/compute	57 ms

Bundle front : 1 294 ko bruts, soit **412 ko gzip**. La valeur dépasse le seuil d'alerte de Vite (500 ko) ; la cause est identifiée (react-simple-maps et world-atlas) et l'axe d'amélioration documenté : chargement différé de la route /carte.

6. Jeu de tests unitaires : le ScoringService

La fonctionnalité couverte est le calcul de recommandation présenté en section 3. Elle est testée par `apps/api/src/scoring/scoring.service.spec.ts`, qui compte dix cas couvrant les filtres éliminatoires, la pondération, les bornes du score et les cas limites.

Le service étant pur — aucune dépendance à simuler — les testsinstancient le module Nest réel et vérifient directement le comportement métier. Deux cas sont représentatifs.

```
// Filtre éliminatoire : un pays sous le niveau de sécurité exigé est exclu (score 0),
// quelle que soit la qualité de ses autres critères
it('should return 0 if safety is too low', () => {
  const result = service.calculateScore(
    makeCriteria({ safety: 2 }),
    { ...baseForm, safety: 3 },
  );
  expect(result).toBe(0);
});

// Pondération : un écart de 1 (sur 4 possibles) sur chacun des 8 niveaux d'activité
// donne 1 - 1/4 = 0,75 par niveau, soit un score global de 75/100
it('should return 75 if all activity levels are half distance apart', () => {
  const result = service.calculateScore(
    makeCriteria({ natureLevel: 4 /* ...les 8 niveaux à 4 */ }),
    { ...baseForm, natureLevel: 5 /* ...les 8 niveaux demandés à 5 */ },
  );
  expect(result).toBe(75);
});
```

Les autres cas couvrent les bornes du score (0 et 100), l'exigence « adapté aux familles » — éliminatoire dans un sens, neutre dans l'autre — et la garantie d'un score entier.

Le harnais anti-régression complet réunit **86 tests API** répartis sur 12 fichiers de spécification (parmi lesquels AuthService : doublon à l'inscription, mauvais mot de passe ; ReviewsService : upsert et rejet 422 sur un pays non visité) et **53 tests front** sur 12 fichiers (Vitest et React Testing Library), soit **139 tests**, tous verts, exécutés à chaque pull request (section 4.1).

7. Mesures de sécurité (OWASP Top 10)

Le mapping complet figure dans docs/securite-accessibilite.md. Voici les points saillants, faille par faille.

Faible OWASP	Mesures mises en œuvre
A01 — Contrôle d'accès	Guards JWT et vérification de propriété (403 sur les ressources d'autrui). Décision documentée sur le champ role : pas encore de routes d'administration.
A02 — Données sensibles	Mots de passe hachés avec bcrypt (12 rounds), secrets hors du dépôt (.env et variables Railway/Netlify), fichier .env.example fourni.
A03 — Injection	Requêtes intégralement paramétrées via Prisma et validation des DTO en entrée. Aucun filtrage de caractères : choix argumenté, un tel filtrage relèverait de la fausse sécurité et affaiblirait les mots de passe (position OWASP et ANSSI).
A04 — Conception	Rate limiting global à 100 requêtes par minute et par IP, renforcé à 5 par minute sur /auth/login et /auth/register.
A05 — Configuration	Helmet, CORS configuré, Swagger désactivé en production, démarrage refusé si JWT_SECRET ou JWT_REFRESH_SECRET est absent.
A06 — Composants vulnérables	Audit npm trié et documenté : 9 vulnérabilités classées « high », liées à d3-color et react-simple-maps, non exploitables dans le contexte et sans downgrade possible. Décision argumentée dans la documentation.
A07 — Authentification	JWT courts (15 minutes) et refresh token (7 jours), politique de mot de passe de 8 caractères minimum dont un chiffre et une majuscule, validée côté front et côté API.
A08 — Intégrité logicielle	Lockfile commité et installation reproductible en CI (npm ci).
A09 — Journalisation	Tentatives de connexion journalisées ; les mots de passe ne le sont jamais.
A10 — SSRF	Non applicable : l'API n'effectue aucune requête sortante pilotée par une entrée utilisateur.

8. Accessibilité

Référentiel retenu : le RGAA (Référentiel Général d'Amélioration de l'Accessibilité). Référentiel officiel français adossé aux WCAG, il permet de justifier des critères précis, là où une checklist qualité généraliste de type OPQUAST reste plus large. La justification complète figure dans docs/securite-accessibilite.md.

8.1 Actions mises en œuvre

- **Carte mondiale accessible au clavier** — les quelque 195 formes SVG, initialement pilotables à la souris uniquement, sont désormais focusables (tabIndex, role="button", aria-label), avec contour de focus visible, tooltip au focus et activation par Entrée ou Espace. Vérifié en conditions réelles.
- **Formulaires** : labels liés aux champs, aria-invalid, aria-describedby et messages en role="alert"
- **Modale de sauvegarde** : piège de focus, fermeture par Échap, restitution du focus à la fermeture
- **Landmarks** (<main>), attribut lang="fr" et hiérarchie de titres cohérente

- **Titre de page unique par route** (RGAA 8.5 et 8.6) via un hook `usePageTitle`, y compris en dynamique sur la fiche pays (« France — CountryTravel »)
- **Contrastes conformes au niveau AA** (RGAA 3.2) : 37 textes informatifs relevés de gray-400 (environ 2,5:1) à gray-500 ou gray-600 (au moins 4,5:1). Seules les icônes décoratives de champs déjà labellisés conservent le gris clair.
- 16 scénarios de recette dédiés (SEC-01 à 07, A11Y-01 à 09), tous rejoués et validés

Limite assumée

Le périmètre RGAA couvert est ciblé sur les parcours du cahier de recettes. Il ne constitue pas un audit exhaustif des 106 critères du référentiel.

9. Cahier de recettes

Le cahier complet réunit **125 scénarios** répartis en 12 sections (authentification, pays, recommandation, sauvegarde, destination aléatoire, carte, navigation, sécurité, accessibilité, visites, avis, tableau de bord). Il est remis avec le code source (`docs/cahier-recettes.md`). Le présent dossier en expose la méthodologie, le bilan d'exécution et un extrait représentatif.

9.1 Méthodologie

Chaque scénario précise ses préconditions, ses étapes, le résultat attendu, le résultat obtenu et son statut. Les scénarios API sont exécutés par requêtes réelles (`curl`) et les scénarios front en navigation réelle.

9.2 Exécution

La campagne complète a été rejouée le 15 juillet 2026, puis complétée à la livraison du sprint 4. Elle a révélé de véritables défauts — BUG-06, BUG-07 et BUG-08, voir section 10 — ce qui démontre que la recette fonctionne comme un outil de détection et non comme une formalité.

Les scénarios couvrent trois natures de tests : fonctionnels (parcours métier — inscription, recommandation, visites, avis, tableau de bord), structurels et de robustesse (validation des bornes, codes ISO invalides, JSON malformé, doublons) et de sécurité (contrôles d'accès 401/403, rate limiting, refus de démarrage sans secret). La dernière campagne affiche 125 scénarios au vert, soit 100 % de réussite.

Bilan par section (125 scénarios, 100 % de réussite) :

#	Section	Scénarios	Réussite
1	Authentification	22	100 %
2	Consultation des pays	14	100 %
3	Moteur de recommandation (calcul + sauvegarde)	20	100 %
4	Destination aléatoire	4	100 %
5	Carte mondiale interactive	6	100 %
6	Navigation générale	3	100 %

#	Section	Scénarios	Réussite
7	Sécurité	7	100 %
8	Accessibilité (RGAA)	9	100 %
9	Pays visités	11	100 %
10	Avis sur les pays (API)	12	100 %
11	Tableau de bord utilisateur	8	100 %
12	Avis sur la fiche pays (front)	9	100 %
Total		125	100 %

9.3 Extrait représentatif

ID	Scénario	Étapes	Résultat attendu	Statut
AUTH-01	Inscription réussie	Envoyer {email, username, password} valides	201, réponse avec user, accessToken et refreshToken	✓
AUTH-21	Mot de passe sans chiffre	Envoyer password: "abccddcsA"	400, « Le mot de passe doit contenir au moins un chiffre »	✓
SEC-02	Rate limiting sur /auth/login	6 requêtes en moins d'une minute, même IP	Les 5 premières traitées, la 6e renvoie 429 Too Many Requests	✓
A11Y-01	Navigation clavier sur la carte	Tab jusqu'à un pays, valider avec Entrée	Focus visible, tooltip identique au survol, navigation vers /pays/:code	✓
CTY-10	Détail d'un pays inexistant	GET /countries/ZZ	404, pas de crash serveur	✓
REC-01	Calcul de recommandation nominal	RecoFormDto complet (valeurs 1–5)	200, top 5 classé par score décroissant	✓
SAV-02	Sauvegarde sans authentification	POST /recommendations/save sans token	401 Unauthorized	✓
RND-04	Erreur réseau au tirage aléatoire	API indisponible, cliquer « Rejouer »	Message role=alert + « Réessayer », pas de page blanche	✓
MAP-03	Navigation depuis la carte	Cliquer sur le Vietnam	Redirection vers /pays/VN	✓
NAV-01	Accès à une URL inconnue	Naviguer vers /url-inexistante	Page 404 dédiée (BUG-06 corrigé)	✓
VIS-02	Ajout d'un pays déjà visité	Renvoyer POST /visits en doublon	409 « Country already marked as visited »	✓

ID	Scénario	Étapes	Résultat attendu	Statut
REV-03	Avis sur un pays non visité	POST /reviews sur un pays non visité	422 « You must have visited this country to review it »	✓
DASH-04	Suppression d'une visite	Cliquer « Retirer » sur un pays	Retrait immédiat de la liste, la carte et le compteur	✓
AVI-05	Modification de mon avis (upsert)	Changer la note, soumettre	Avis mis à jour sans doublon (même id)	✓

10. Plan de correction des bogues

Le plan complet réunit **11 fiches** (BUG-01 à BUG-11), chacune détaillant le symptôme, l'analyse de cause, la correction apportée, le commit de référence et le test de non-régression associé. Il est remis avec le code source (docs/plan-correction-bogues.md). Le présent dossier en expose le processus, une synthèse des 11 fiches et une fiche complète à titre d'exemple.

10.1 Processus de traitement

- 9. Détection** — scénario de recette en échec, retour utilisateur ou échec de la CI
- 10. Consignation** — ouverture d'une fiche : contexte, comportement observé face au comportement attendu, gravité (bloquant, majeur ou mineur)
- 11. Analyse** — identification de la cause racine dans le code
- 12. Correction** — branche dédiée (fix/CT-XXX/n ou BUG-XX) puis pull request
- 13. Vérification** — rejeu du scénario de recette concerné et CI verte avant merge, avec ajout d'un test de non-régression lorsque c'est pertinent

10.2 Synthèse des anomalies traitées

Les onze anomalies sont corrigées et couvertes par un rejeu de recette, complété d'un test de non-régression lorsque c'était pertinent.

ID	Titre	Gravité	Origine
BUG-01	Crash sur JSON invalide en cache	Bloquant	Front — recommandation
BUG-02	Token non persisté en localStorage	Majeur	Front — authentification
BUG-03	Perte de contexte au retour arrière	Majeur	Front — recommandation
BUG-04	404 sur les URLs directes (SPA Netlify)	Bloquant	Front — déploiement
BUG-05	Échec de lint en CI	Mineur	Front — CI/CD
BUG-06	Page blanche sur URLs inconnues	Majeur	Front — routage
BUG-07	Message erroné sur la longueur du mot de passe	Mineur	API — validation
BUG-08	Seed inopérant (API externe dépréciée)	Bloquant	API — données

ID	Titre	Gravité	Origine
BUG-09	Sauvegarde de recommandation sans retour utilisateur	Majeur	Front — recommandation
BUG-10	Expiration de session silencieuse (401 muets)	Majeur	Front — authentification
BUG-11	Secret de refresh non validé au démarrage	Mineur	API — authentification

10.3 Exemple de fiche

BUG-08 — Seed inopérant : dépendance à une API externe dépréciée (bloquant pour toute nouvelle installation)

- **Symptôme** — sur une base vierge, le seed échouait avec l'erreur `TypeError: all.filter is not a function`. Détecté lors du test d'installation à froid du projet.
- **Cause racine** — `prisma/seed.ts` appelait l'API `restcountries.com/v3.1` au moment du seed ; cette version de l'API a été coupée et renvoie désormais un objet d'erreur. La base de production, alimentée avant la coupure, masquait l'anomalie.
- **Correctif** — suppression de la dépendance réseau : les données ont été exportées vers un fichier versionné `prisma/data/countries.json` et le seed réécrit pour le lire. Le seed est désormais déterministe, reproductible et hors ligne.
- **Vérification** — `npm run db:setup` sur une base Docker vierge insère les 30 pays et l'application complète fonctionne sur cette base fraîche.

Cette fiche illustre l'intérêt du processus : une anomalie invisible en production, révélée uniquement parce que le manuel de déploiement a été testé en conditions réelles.

11. Historique des versions

Le flux de travail associe une branche à chaque ticket ou bogue (CT-xxx, BUG-xx), une pull request, une CI verte obligatoire, puis un merge sur main qui déclenche le déploiement. À la date de rédaction, le dépôt compte **118 commits et 32 pull requests**.

Jalon	Contenu livré	Pull requests
Sprint 1	Socle technique (NestJS, React, Prisma, CI/CD) et authentification	#1 à #3
Sprint 2	Catalogue pays, seed, moteur de scoring, fiches pays	#3 à #8
Sprint 3	Recommandation multi-étapes, sauvegarde, destination aléatoire, carte mondiale	#9 à #19
Qualité	Tests front, sécurité et accessibilité (C2.2.3), documentation, qualité et performance (C2.1.1)	#17 à #23
Sprint 4	Pays visités, avis, tableau de bord	#24 à #26
Corrections	BUG-09, BUG-10, BUG-11	#27 à #32

12. Manuels

Les trois manuels figurent ici en version condensée ; les versions intégrales sont remises avec le code source (docs/manuel-*.md).

12.1 Manuel de déploiement

Prérequis : Node 20 ou supérieur. Installation locale recommandée, avec une base PostgreSQL lancée via Docker :

```
git clone <url-du-repo> && cd countryTravel
docker compose up -d                    # PostgreSQL 17 en local
cp apps/api/.env.example apps/api/.env
npm install                             # workspaces + client Prisma
npm run db:setup                         # migrations + seed (30 pays)
npm run dev                             # API :3000 + front :5173
```

JWT_SECRET et JWT_REFRESH_SECRET doivent être **deux valeurs distinctes** — l'API refuse de démarrer si l'une manque. Les générer avec openssl rand -base64 64, lancé deux fois (ou l'équivalent Node sous Windows). Deux autres parcours d'installation sont documentés : application déjà déployée, et base PostgreSQL sans Docker.

En production : npm run build:api && npm run build:front avec NODE_ENV=production (ce qui désactive Swagger). Le déploiement continu est assuré par GitHub Actions à chaque merge sur main. Le manuel se termine par une checklist post-déploiement. Il a été validé par une installation à froid sur base vierge — test qui a lui-même révélé et fait corriger BUG-08.

12.2 Manuel d'utilisation

L'application est utilisable **sans compte** pour toute la consultation et la recommandation. Un compte n'est requis que pour sauvegarder ses recommandations et accéder au tableau de bord.

- **Explorer** — liste des pays avec recherche et filtres par continent ou monnaie, puis fiche pays détaillée.
- **Recommandation** — questionnaire multi-étapes (chaque critère de « Pas du tout » à « Énormément », plus un interrupteur famille) aboutissant à un top 5 assorti d'un score de compatibilité.
- **Carte mondiale** — pays colorés selon leur note moyenne ; survol pour le détail, clic pour la fiche, zoom avec Ctrl et molette. Entièrement navigable au clavier (Tab et Entrée).
- **Sécurité perçue** — après 5 tentatives de connexion en moins d'une minute, un blocage temporaire s'applique (« Too Many Requests ») ; la session expire au bout de 15 minutes et une reconnexion est demandée.

Le manuel détaille également les fonctions d'accessibilité (clavier, lecteurs d'écran) et propose un tableau de résolution des problèmes courants.

12.3 Manuel de mise à jour

Livrer une modification : créer une branche depuis main, ouvrir une pull request (la CI pr.yml exécute lint, tests et build ; la fusion n'est possible que si tout est vert), merger, le déploiement se déclenche automatiquement.

Dépendances : npm audit et npm audit fix, **jamais --force sans analyse** (montées de version majeures non maîtrisées). Après toute mise à jour : npx tsc --noEmit puis la suite de tests, avant commit.

Évolution de schéma : `npx prisma migrate dev --name <desc>` en local, avec commit du dossier de migration généré. En production, cette étape est **manuelle** car hors pipeline :

```
cd apps/api
DATABASE_URL="<url-railway>" npx prisma migrate deploy
```

Retour arrière : `git revert`, jamais de `push --force`. Un rollback en un clic est également disponible depuis les tableaux de bord Railway et Netlify. L'historique des versions est porté par les merge commits de main — une pull request correspond à une évolution — et par les fiches du plan de correction des bogues.

13. Axes d'amélioration et perspectives

13.1 Axes techniques identifiés

- **Automatiser l'application des migrations Prisma** dans le pipeline `deploy-api.yml`, aujourd'hui étape manuelle en production
- **Renforcer la couverture de tests des composants d'interface** côté front, actuellement couverts par la recette manuelle
- **Alléger le bundle de la carte mondiale** par un chargement différé de la route `/carte` (`react-simple-maps` et `world-atlas`)
- **Harmoniser l'organisation des pages** sur le découpage par feature adopté partout ailleurs

13.2 Perspectives produit

- Module de profil utilisateur et gestion de la conformité RGPD (CT-023)
- Ouverture complète du volet communautaire (avis, amis) pour donner toute sa valeur à la carte notée
- Authentification via des fournisseurs externes (OAuth)
- Supervision et alerting en production, qui relèvent du Bloc 4 « maintien en condition opérationnelle »

Annexe — Documents complets remis avec le code source

Ce dossier condense chaque élément attendu. Les versions intégrales sont remises avec le code source, dans le dossier `docs/` du dépôt.

Document	Fichier
Cahier de recettes (125 scénarios)	<code>docs/cahier-recettes.md</code>
Plan de correction des bogues (11 fiches)	<code>docs/plan-correction-bogues.md</code>
Sécurité et accessibilité	<code>docs/securite-accessibilite.md</code>
Critères de qualité et de performance	<code>docs/qualite-performance.md</code>
Manuel de déploiement	<code>docs/manuel-deploiement.md</code>
Manuel d'utilisation	<code>docs/manuel-utilisation.md</code>
Manuel de mise à jour	<code>docs/manuel-mise-a-jour.md</code>